Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «АПО ЭВМиС»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel - обработка символьных данных»

Выполнил:
Студент 4-го курса
Группы АС-66
Ярома А. И.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы

Изучить программную модель процессора Intel и закрепить работу с регистровой и оперативной памятью при обработке символьной последовательности на ассемблере.

Задание. Вариант 12

Дан непустой текст длиной не более 100 символов. Ввод завершается точкой, которая в текст не включается. Проверить, равно ли количество строчных русских букв количеству цифр. Если условие истинно, циклически сдвинуть текст на три позиции влево и напечатать результат. Иначе вывести сообщение об отсутствии преобразования.

Особенности реализации

Программа состоит из последовательных этапов: ввод, проверка допустимости текста, проверка условия, преобразование и печать. Все этапы реализованы на MASM x86: ReadText, CheckConditionAsm, RotateLeftAsm и PrintText. Точка входа start вызывает ProgramMain. Исходников C/C++ и зависимости от CRT нет; ввод и вывод используют Win32 API.

В рабочем буфере один DWORD представляет один символ Unicode. Так русская буква считается одним символом и сдвиг не разрывает многобайтовую кодировку. Буфер имеет постоянную ёмкость 101 элемент и расположен в локальном кадре ProgramMain. Все изменяемые данные локальны; в .const находятся только неизменяемые сообщения. Дополнительного массива для сдвига нет. Вспомогательные буферы символа и числа имеют фиксированный размер.

Проверка учитывает а-я (0430h-044Fh), ё (0451h) и цифры 0-9. Прописные русские и латинские буквы не увеличивают счётчик строчных русских букв. Равенство 0 = 0 удовлетворяет условию.

Эффективное число сдвигов равно 3 mod n. На каждом шаге первый символ сохраняется в EAX, команда REP MOVSD переносит оставшиеся элементы влево, затем сохранённый символ записывается в конец. CLD задаёт движение по возрастающим адресам. Дополнительная память O(1); время преобразования O(n).

Текст программы

Проверка и преобразование: text_processing.asm

```
; APO EVMiS, lab 2, variant 12. MASM, x86, cdecl.
; Each DWORD stores one Unicode character. No global variables.
.386
.model flat, C
.code
; int CheckConditionAsm(text, length, letters, digits)
; Counts Russian lowercase letters (including yo) and ASCII digits.
; EAX = 1 when the counts are equal, otherwise EAX = 0.
CheckConditionAsm PROC USES esi ebx,
    pText:PTR DWORD, charCount:DWORD, pLetters:PTR DWORD, pDigits:PTR DWORD
    mov esi, pText
    mov ecx, charCount
    xor ebx, ebx
    xor edx, edx
    cld
    test ecx, ecx
    jz counts_ready
check_character:
    lodsd
    cmp eax, 0430h              ; Russian lowercase a
    jb check_yo
    cmp eax, 044Fh              ; Russian lowercase ya
    jbe count_letter
check_yo:
    cmp eax, 0451h              ; Russian lowercase yo
    je count_letter
    cmp eax, '0'
    jb next_character
    cmp eax, '9'
    ja next_character
    inc edx
    jmp next_character
count_letter:
    inc ebx
next_character:
    dec ecx
    jnz check_character
counts_ready:
    mov eax, pLetters
    mov [eax], ebx
    mov eax, pDigits
    mov [eax], edx
    xor eax, eax
    cmp ebx, edx
    sete al
    ret
CheckConditionAsm ENDP
; void RotateLeftAsm(text, length)
; Three left rotations; effective count is 3 mod length.
; A character is kept in EAX, the remaining characters move with REP MOVSD.
; No second text buffer and no pushes inside the rotation loop.
RotateLeftAsm PROC USES esi edi ebx,
    pText:PTR DWORD, charCount:DWORD
    cld
    mov ebx, charCount
    cmp ebx, 1
    jbe rotation_done
    mov eax, 3
    xor edx, edx
    div ebx
    mov ebx, edx                ; number of single-position rotations
    test ebx, ebx
    jz rotation_done
rotate_once:
    mov edi, pText
    mov eax, [edi]              ; retain the first character
    lea esi, [edi + 4]
    mov ecx, charCount
```

```
74      dec ecx
75      rep movsd                ; move text[1..n-1] to text[0..n-2]
76      mov [edi], eax            ; append the retained character
77      dec ebx
78      jnz rotate_once
80  rotation_done:
81      ret
82  RotateLeftAsm ENDP
83  END
```

Ввод, вывод и точка входа: main.asm

```
 1  ; Lab 2, variant 12. All executable program logic is MASM x86.
 2  ; Win32 API supplies console/file I/O; neither C++ nor a CRT is linked.
 3  .386
 4  .model flat, stdcall
 5  option casemap:none
 7  MAX_LENGTH EQU 100
 8  INPUT_OK EQU 0
 9  INPUT_EMPTY EQU 1
10  INPUT_LONG EQU 2
11  INPUT_NO_PERIOD EQU 3
12  INPUT_BAD_ENCODING EQU 4
13  CHAR_EOF EQU -1
14  CHAR_INVALID EQU -2
16  GetStdHandle PROTO :DWORD
17  GetConsoleMode PROTO :DWORD, :PTR DWORD
18  SetConsoleOutputCP PROTO :DWORD
19  ReadFile PROTO :DWORD, :DWORD, :DWORD, :PTR DWORD, :DWORD
20  ReadConsoleW PROTO :DWORD, :DWORD, :DWORD, :PTR DWORD, :DWORD
21  WriteFile PROTO :DWORD, :DWORD, :DWORD, :PTR DWORD, :DWORD
22  ExitProcess PROTO :DWORD
23  CheckConditionAsm PROTO C :PTR DWORD, :DWORD, :PTR DWORD, :PTR DWORD
24  RotateLeftAsm PROTO C :PTR DWORD, :DWORD
26  InputContext STRUCT
27      handle DWORD ?
28      console DWORD ?
29      mode DWORD ?
30      skipLF DWORD ?
31  InputContext ENDS
33  ; Immutable messages, not global variables. Source file encoding is UTF-8.
34  .const
35  msgHeader BYTE 'АПО ЭВМиС. Лабораторная работа №2',13,10
36      BYTE 'Ярома А. И. | АС-66 | Вариант 12 | MASM x86',13,10
37      BYTE 'Условие: число строчных русских букв равно числу цифр.',13,10
38      BYTE 'Правило: циклический сдвиг на 3 символа влево.',13,10
39      BYTE 'Введите от 1 до 100 символов; завершите точкой и нажмите Enter:',13,10,0
40  msgOriginal BYTE 13,10,'Исходный текст: "',0
41  msgLength BYTE '"',13,10,'Длина: ',0
42  msgCharacters BYTE ' символов',13,10,0
43  msgLetters BYTE 'Строчных русских букв: ',0
44  msgDigits BYTE 13,10,'Цифр: ',0
45  msgFalse BYTE 13,10,'Условие не выполнено. Текст не преобразуется.',13,10,0
46  msgTrue BYTE 13,10,'Условие выполнено. Сдвиг на 3 символа влево.',13,10,0
47  msgResult BYTE 'Результат: "',0
48  msgEnd BYTE '"',13,10,0
49  msgInvalid BYTE 'Последовательность не является допустимым текстом: ',0
50  msgEmpty BYTE 'пустой текст.',13,10,0
51  msgLong BYTE 'больше 100 символов.',13,10,0
52  msgNoPeriod BYTE 'нет завершающей точки.',13,10,0
53  msgEncoding BYTE 'ошибка кодировки Unicode.',13,10,0
54  msgNotConverted BYTE 'Преобразование не выполняется.',13,10,0
56  .code
58  ; Read one UTF-16 code unit from a console, or one byte from a file/pipe.
59  ; Fixed-size locals only; EAX = unit or CHAR_EOF.
60  ReadRaw PROC USES esi, pContext:PTR InputContext
61      LOCAL unitValue:DWORD, received:DWORD
62      mov esi, pContext
63      mov unitValue, 0
64      cmp [esi].InputContext.console, 0
65      je raw_file
66      invoke ReadConsoleW, [esi].InputContext.handle, ADDR unitValue, 1, ADDR received, 0
67      jmp raw_done
```

```
 68  raw_file:
 69      invoke ReadFile, [esi].InputContext.handle, ADDR unitValue, 1, ADDR received, 0
 70  raw_done:
 71      test eax, eax
 72      jz raw_eof
 73      cmp received, 0
 74      je raw_eof
 75      mov eax, unitValue
 76      ret
 77  raw_eof:
 78      mov eax, CHAR_EOF
 79      ret
 80  ReadRaw ENDP
 82  ; Decode one Unicode scalar. Console input is UTF-16; redirected input is UTF-8.
 83  ; EAX = scalar, CHAR_EOF, or CHAR_INVALID. No text-sized temporary buffer.
 84  ReadCharacter PROC USES esi ebx edi, pContext:PTR InputContext
 85      LOCAL minimum:DWORD
 86      mov esi, pContext
 87      invoke ReadRaw, esi
 88      cmp eax, CHAR_EOF
 89      je character_return
 90      cmp [esi].InputContext.console, 0
 91      je decode_utf8
 92      cmp eax, 0D800h
 93      jb character_return
 94      cmp eax, 0DBFFh
 95      ja check_low_surrogate
 96      sub eax, 0D800h
 97      shl eax, 10
 98      mov ebx, eax
 99      invoke ReadRaw, esi
100      cmp eax, 0DC00h
101      jb character_invalid
102      cmp eax, 0DFFFh
103      ja character_invalid
104      sub eax, 0DC00h
105      lea eax, [eax + ebx + 10000h]
106      ret
107  check_low_surrogate:
108      cmp eax, 0DFFFh
109      jbe character_invalid
110      ret
111  decode_utf8:
112      cmp eax, 80h
113      jb character_return
114      mov minimum, 80h
115      mov edi, 1
116      cmp eax, 0C2h
117      jb character_invalid
118      cmp eax, 0DFh
119      jbe utf8_two
120      mov minimum, 800h
121      mov edi, 2
122      cmp eax, 0EFh
123      jbe utf8_three
124      cmp eax, 0F4h
125      ja character_invalid
126      mov minimum, 10000h
127      mov edi, 3
128      and eax, 7
129      jmp utf8_continue
130  utf8_three:
131      and eax, 0Fh
132      jmp utf8_continue
133  utf8_two:
134      and eax, 1Fh
135  utf8_continue:
136      mov ebx, eax
137  utf8_next:
138      invoke ReadRaw, esi
139      mov edx, eax
140      and edx, 0FFFFFFC0h
141      cmp edx, 80h
142      jne character_invalid
```

```
    shl ebx, 6
    and eax, 3Fh
    or ebx, eax
    dec edi
    jnz utf8_next
    mov eax, ebx
    cmp eax, minimum
    jb character_invalid
    cmp eax, 10FFFFh
    ja character_invalid
    cmp eax, 0D800h
    jb character_return
    cmp eax, 0DFFFh
    jbe character_invalid
character_return:
    ret
character_invalid:
    mov eax, CHAR_INVALID
    ret
ReadCharacter ENDP
; Stage 1: read/validate. The point is not stored; CRLF is one newline.
; The sole text buffer has a constant capacity of MAX_LENGTH + 1 DWORDs.
ReadText PROC USES esi edi ebx, pContext:PTR InputContext, pText:PTR DWORD, pLength:PTR DWORD
    LOCAL overflow:DWORD
    mov esi, pContext
    mov edi, pText
    xor ebx, ebx
    mov overflow, 0
read_next:
    invoke ReadCharacter, esi
    cmp eax, CHAR_EOF
    je input_no_period
    cmp eax, CHAR_INVALID
    je input_bad_encoding
    cmp [esi].InputContext.skipLF, 0
    je check_newline
    mov [esi].InputContext.skipLF, 0
    cmp eax, 10
    je read_next
check_newline:
    cmp eax, 13
    jne check_period
    mov [esi].InputContext.skipLF, 1
    mov eax, 10
check_period:
    cmp eax, '.'
    je input_finished
    cmp ebx, MAX_LENGTH
    jae input_overflow
    mov [edi + ebx * 4], eax
    inc ebx
    jmp read_next
input_overflow:
    mov overflow, 1
    jmp read_next
input_finished:
    mov eax, INPUT_LONG
    cmp overflow, 0
    jne input_return
    mov eax, INPUT_EMPTY
    test ebx, ebx
    jz input_return
    mov eax, INPUT_OK
    jmp input_return
input_no_period:
    mov eax, INPUT_NO_PERIOD
    jmp input_return
input_bad_encoding:
    mov eax, INPUT_BAD_ENCODING
input_return:
    mov DWORD PTR [edi + ebx * 4], 0
    mov edx, pLength
    mov [edx], ebx
    ret
```

```
218 ReadText ENDP
220 ; Write all bytes, including the case of a partial WriteFile.
221 WriteAll PROC USES esi ebx, hOutput:DWORD, pBytes:DWORD, byteCount:DWORD
222     LOCAL written:DWORD
223     mov esi, pBytes
224     mov ebx, byteCount
225 write_more:
226     test ebx, ebx
227     jz write_success
228     invoke WriteFile, hOutput, esi, ebx, ADDR written, 0
229     test eax, eax
230     jz write_return
231     cmp written, 0
232     je write_failed
233     add esi, written
234     sub ebx, written
235     jmp write_more
236 write_failed:
237     xor eax, eax
238 write_return:
239     ret
240 write_success:
241     mov eax, 1
242     ret
243 WriteAll ENDP
245 WriteMessage PROC hOutput:DWORD, pMessage:DWORD
246     mov edx, pMessage
247     xor ecx, ecx
248 message_length:
249     cmp BYTE PTR [edx + ecx], 0
250     je message_write
251     inc ecx
252     jmp message_length
253 message_write:
254     invoke WriteAll, hOutput, edx, ecx
255     ret
256 WriteMessage ENDP
258 ; Encode one scalar in a fixed four-byte local, then print it.
259 PrintCharacter PROC USES ebx, hOutput:DWORD, value:DWORD
260     LOCAL encoded:DWORD, byteCount:DWORD
261     mov eax, value
262     lea edx, encoded
263     mov byteCount, 1
264     cmp eax, 80h
265     jb encode_last
266     mov byteCount, 2
267     cmp eax, 800h
268     jb encode_continuations
269     mov byteCount, 3
270     cmp eax, 10000h
271     jb encode_continuations
272     mov byteCount, 4
273 encode_continuations:
274     mov ecx, byteCount
275     dec ecx
276 encode_tail:
277     mov ebx, eax
278     and ebx, 3Fh
279     or bl, 80h
280     mov [edx + ecx], bl
281     shr eax, 6
282     dec ecx
283     jnz encode_tail
284     cmp byteCount, 2
285     je encode_head_two
286     cmp byteCount, 3
287     je encode_head_three
288     or eax, 0F0h
289     jmp encode_store_head
290 encode_head_three:
291     or eax, 0E0h
292     jmp encode_store_head
293 encode_head_two:
294     or eax, 0C0h
```

```
295 encode_store_head:
296     mov [edx], al
297     jmp encode_write
298 encode_last:
299     mov [edx], al
300 encode_write:
301     invoke WriteAll, hOutput, ADDR encoded, byteCount
302     ret
303 PrintCharacter ENDP
305 ; Stage 4: print the transformed text, without changing the working buffer.
306 PrintText PROC USES esi ebx, hOutput:DWORD, pText:PTR DWORD, charCount:DWORD
307     mov esi, pText
308     mov ebx, charCount
309 print_next:
310     test ebx, ebx
311     jz print_done
312     invoke PrintCharacter, hOutput, DWORD PTR [esi]
313     add esi, 4
314     dec ebx
315     jmp print_next
316 print_done:
317     ret
318 PrintText ENDP
320 PrintUnsigned PROC USES ebx edi, hOutput:DWORD, value:DWORD
321     LOCAL digits[10]:BYTE
322     lea edi, digits[10]
323     mov eax, value
324     mov ebx, 10
325 number_next:
326     xor edx, edx
327     div ebx
328     add dl, '0'
329     dec edi
330     mov [edi], dl
331     test eax, eax
332     jnz number_next
333     lea ecx, digits[10]
334     sub ecx, edi
335     invoke WriteAll, hOutput, edi, ecx
336     ret
337 PrintUnsigned ENDP
339 ProgramMain PROC
340     LOCAL text[MAX_LENGTH + 1]:DWORD
341     LOCAL context:InputContext
342     LOCAL hOutput:DWORD, charLength:DWORD, letters:DWORD, digits:DWORD, status:DWORD
343     invoke GetStdHandle, -10
344     mov context.handle, eax
345     invoke GetConsoleMode, context.handle, ADDR context.mode
346     mov context.console, eax
347     mov context.skipLF, 0
348     invoke GetStdHandle, -11
349     mov hOutput, eax
350     invoke SetConsoleOutputCP, 65001
351     invoke WriteMessage, hOutput, ADDR msgHeader
352     invoke ReadText, ADDR context, ADDR text, ADDR charLength
353     mov status, eax
354     cmp eax, INPUT_OK
355     jne main_invalid
356     invoke WriteMessage, hOutput, ADDR msgOriginal
357     invoke PrintText, hOutput, ADDR text, charLength
358     invoke WriteMessage, hOutput, ADDR msgLength
359     invoke PrintUnsigned, hOutput, charLength
360     invoke WriteMessage, hOutput, ADDR msgCharacters
361     ; Stage 2: separate condition procedure.
362     invoke CheckConditionAsm, ADDR text, charLength, ADDR letters, ADDR digits
363     mov status, eax
364     invoke WriteMessage, hOutput, ADDR msgLetters
365     invoke PrintUnsigned, hOutput, letters
366     invoke WriteMessage, hOutput, ADDR msgDigits
367     invoke PrintUnsigned, hOutput, digits
368     cmp status, 0
369     je main_false
370     invoke WriteMessage, hOutput, ADDR msgTrue
371     ; Stage 3: separate in-place conversion procedure, uses REP MOVSD.
```

```
372      invoke RotateLeftAsm, ADDR text, charLength
373      invoke WriteMessage, hOutput, ADDR msgResult
374      invoke PrintText, hOutput, ADDR text, charLength
375      invoke WriteMessage, hOutput, ADDR msgEnd
376      xor eax, eax
377      ret
378  main_false:
379      invoke WriteMessage, hOutput, ADDR msgFalse
380      xor eax, eax
381      ret
382  main_invalid:
383      invoke WriteMessage, hOutput, ADDR msgInvalid
384      mov edx, OFFSET msgEncoding
385      cmp status, INPUT_EMPTY
386      je main_empty
387      cmp status, INPUT_LONG
388      je main_long
389      cmp status, INPUT_NO_PERIOD
390      je main_no_period
391      jmp main_error_message
392  main_empty:
393      mov edx, OFFSET msgEmpty
394      jmp main_error_message
395  main_long:
396      mov edx, OFFSET msgLong
397      jmp main_error_message
398  main_no_period:
399      mov edx, OFFSET msgNoPeriod
400  main_error_message:
401      invoke WriteMessage, hOutput, edx
402      invoke WriteMessage, hOutput, ADDR msgNotConverted
403      mov eax, 1
404      ret
405  ProgramMain ENDP
407  start:
408      invoke ProgramMain
409      invoke ExitProcess, eax
410  END start
```

Результаты выполнения

Скриншоты получены при запуске собранной программы в конфигурации Release / x86. Ввод передан из файлов UTF-8; исходная последовательность видна в выводе программы.

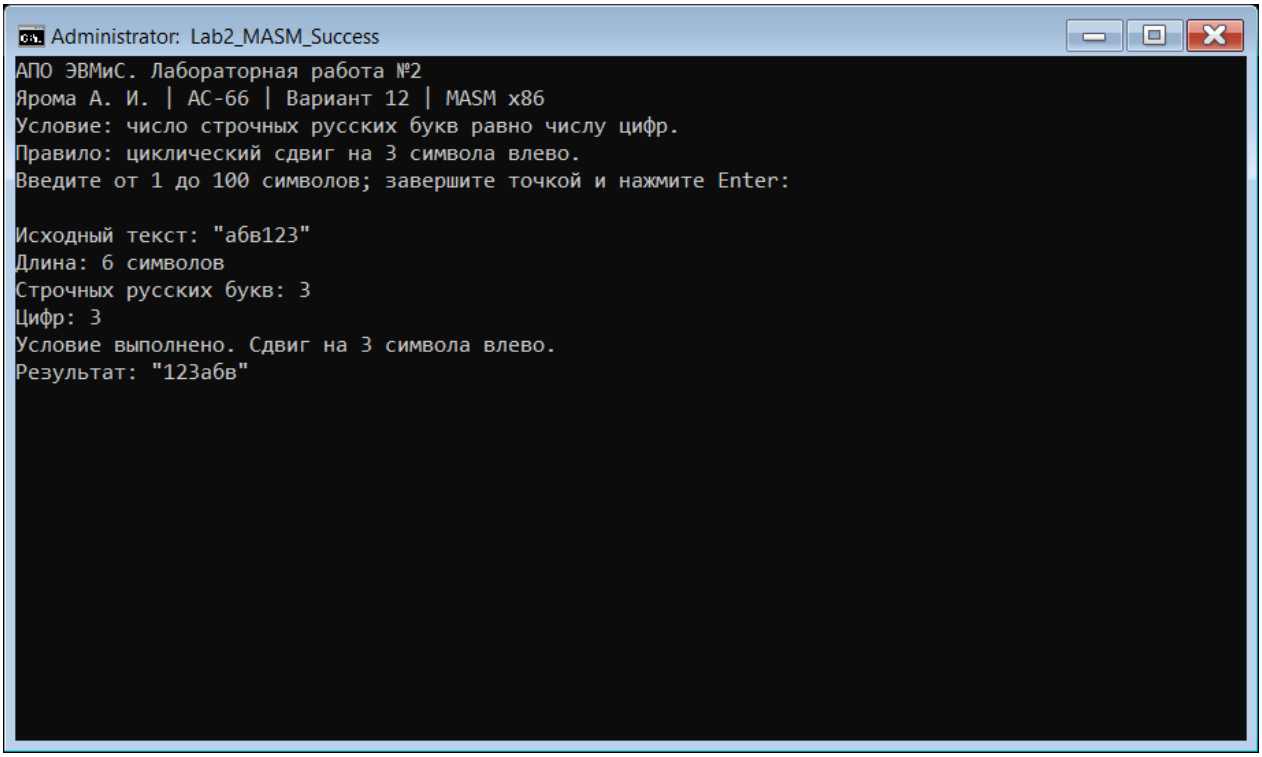


Рисунок 1 - «абв123.»: три буквы и три цифры; результат «123абв».

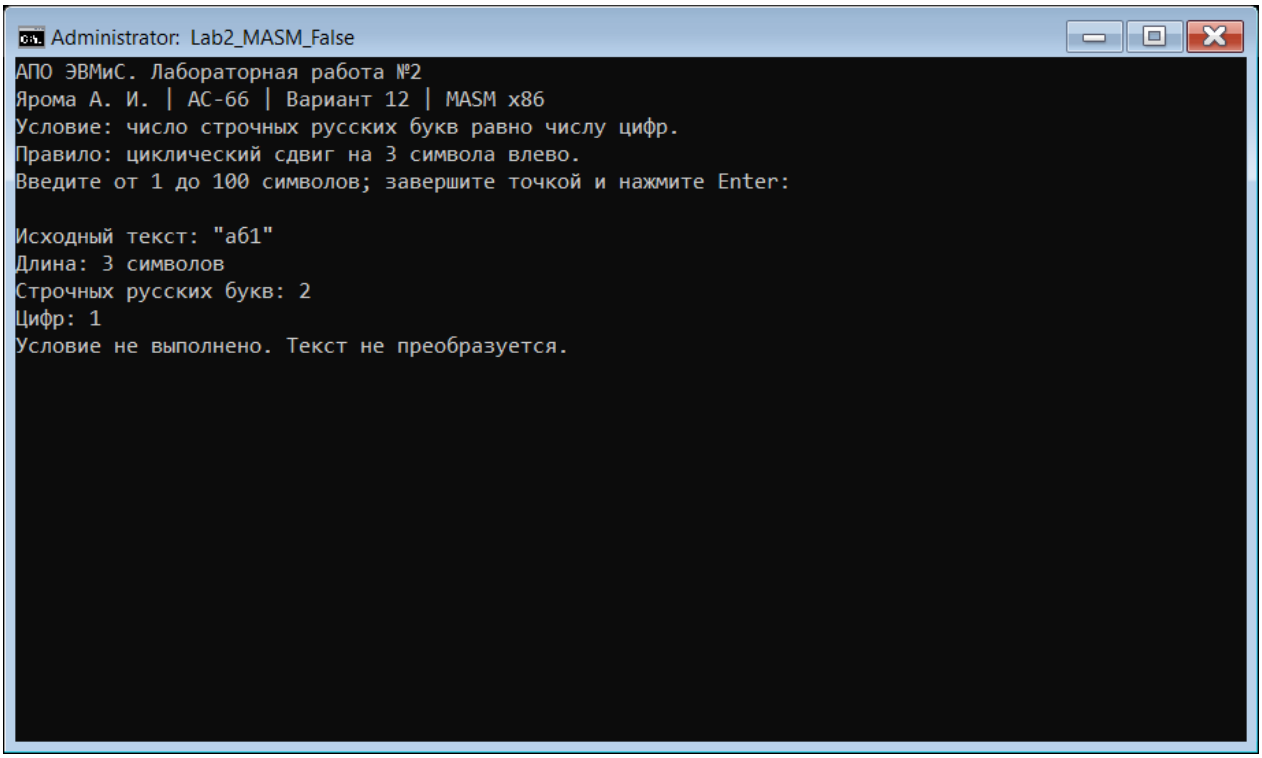


Рисунок 2 - «аб1.»: две буквы и одна цифра; преобразования нет.

Граничные случаи и проверка

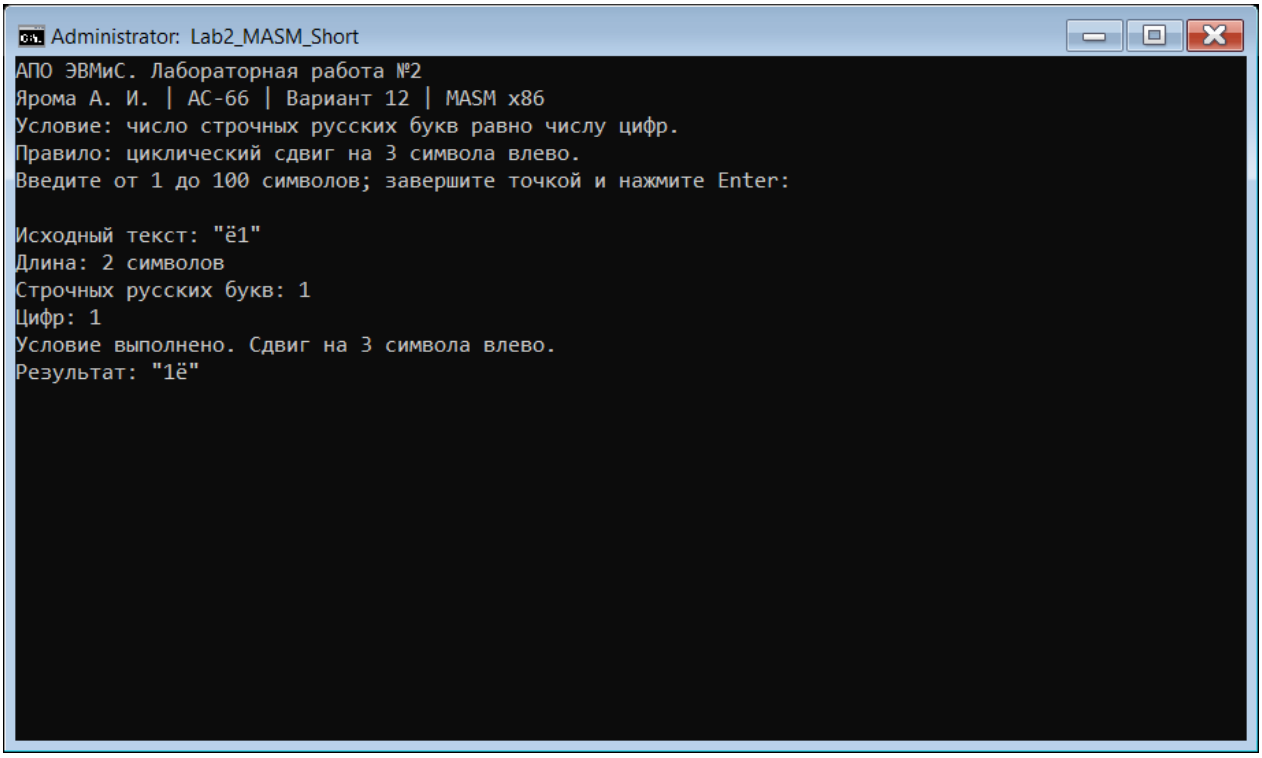


Рисунок 3 - «ё1.»: 3 mod 2 = 1; результат «1ё». Буква ё учтена.

Ввод	Ожидаемое поведение
ABC.	0 букв = 0 цифр; длина 3, результат ABC
AB.	Сдвиг на одну позицию; результат BA
.	Пустой текст; преобразование не выполняется
101 символ и точка	Превышение лимита; преобразования нет
abc без точки	Нет завершающей точки; преобразования нет
100 символов и точка	Допустимая максимальная длина

В Debug и Release прошли по 366 сценариев ввода: обычные и граничные тексты, 300 случайных строк, 20 ошибочных последовательностей и проверки окончания на точке и CRLF. Дополнительно прошли 14 тестов настоящего консольного ввода ReadConsoleW и 1010 прямых тестов процедур для длин от 0 до 100. Проверены оба счётчика, неизменность буфера после проверки условия, результат сдвига и сохранность контрольных элементов за границами текста. Всего 1756 сценариев. Аудит PE: импортируется только KERNEL32.dll, изменяемых секций .data/.bss нет. Перенос строки до точки входит в текст, CRLF считается одним символом.

Вывод

Разработана полностью ассемблерная программа варианта 12: процедура сравнивает число строчных русских букв и цифр, а при истинном условии другая процедура циклически сдвигает текст влево на три позиции. Использована строковая команда REP MOVSD. Проверки подтвердили корректную обработку русских букв, коротких строк и предельной длины, а также отказ от преобразования при недопустимом вводе.